Alexandria University
faculty of Engineering
Artificial intelligence

2025 - 2026

ASSIGEMENT 1

Report

Made by:

Abdelrhman Ahmed Anwar
Id : 8733
Ahmed Tamer
id: 8842
Taha Mostafa Mohamed
id: 9539

121938

Assignment 1 Report

1. Problem statement

The 8-puzzle is a sliding puzzle with 8 numbered tiles and a blank space (represented by 0) on a 3×3 board. The objective is to reach the goal configuration [0,1,2,3,4,5,6,7,8] by swapping the blank with adjacent tiles (Up/Down/Left/Right). Cost per move = 1.

2. Implemented algorithms

- **Uninformed search**: BFS, DFS (and Iterative DFS)
- **Informed search**: A* using two heuristics:
  - Manhattan distance (sum of |dx| + |dy| for all tiles to their goal positions).
  - Euclidean distance (sum of Euclidean distances from tile current to goal positions).

3. Data structures

- **Permutation encoding**: I encode every board permutation as a unique integer in range [0, 9! - 1] using factoradic code:
  - get_state(grid) maps a 3×3 list into an integer.
  - get_grid(state_number) maps the integer back to a grid.
  - This encoding lets us use arrays/lists indexed by state-id (parent array par) and compactly represent explored states.
- **Open frontier / queue**:
  - BFS uses a FIFO Queue.
  - DFS uses a stack (Python list append/pop).
  - A* uses a min-heap (priority queue via heapq) keyed by f = g + h.
- **Auxiliary**:
  - par (parent array indexed by state id) to reconstruct path.
  - g_cost dict for the A* g-values (cost-from-start).

4. Implementation details & corrections

- Implemented get_state and get_grid using factoradic encoding so each permutation has unique id (0..362879).

- get_neighbours(state) returns all legal successor states by swapping the blank with valid adjacent tiles.
- A* checks g_cost and pushes (f_cost, neighbor) to the heap; par is updated for path

5. Run sample Results

Strat with easy(initial state) [1,2,3,4,0,5,6,7,8]

Algorithm	Cost of path (steps)	Nodes expanded	Search depth	Running time (s)
A* (Manhattan)	*14*	*98*	*14*	*0.000887*
A* (Euclidean)	*14*	*130*	*14*	*0.000998*
BFS	*14*	*3718*	*14*	*0.023997*
DFS (limited)	*53436*	*129832*	*53436*	*0.601604*

Strat with high difficulty (initial state) [8,7,6,5,4,3,2,1,0]

Algorithm	Cost of path (steps)	Nodes expanded	Search depth	Running time (s)
A* (Manhattan)	*28*	180	*28*	*0.001000*
A* (Euclidean)	28	5113	*28*	*0.055511*
BFS	28	177224	*28*	*1.131921*
DFS (limited)	64830	106784	64830	*0.518691*

5.2 Sample run (start = [1,2,3,4,0,5,6,7,8]) — A* Manhattan

```
A* (Manhattan) solution:
do you want interactive steps? (y/n): n
Full Steps:
Steps:
 1  2  3
 4  x  5
 6  7  8

 1  2  3
 x  4  5
 6  7  8

 x  2  3
 1  4  5
 6  7  8

 2  x  3
 1  4  5
 6  7  8

 2  3  x
 1  4  5
 6  7  8

 2  3  5
 1  4  x
 6  7  8

 2  3  5
 1  x  4
 6  7  8

 2  x  5
 1  3  4
 6  7  8

 x  2  5
 1  3  4
 6  7  8
```

```
 x  2  5
 1  3  4
 6  7  8

 1  2  5
 x  3  4
 6  7  8

 1  2  5
 3  x  4
 6  7  8

 1  2  5
 3  4  x
 6  7  8

 1  2  x
 3  4  5
 6  7  8

 1  x  2
 3  4  5
 6  7  8

 x  1  2
 3  4  5
 6  7  8

A* (Manhattan) Path length: 15
A* (Manhattan) Nodes expanded: 98
```

6. Discussion & comparison

- The Manhattan heuristic is admissible and consistent for the sliding puzzle; it often produces fewer expansions than Euclidean in practice here (example start2 shows Manhattan expanding far fewer nodes than Euclidean), so Manhattan is preferable for A* on 8-puzzle.
- BFS guarantees shortest path but can be much less memory/time efficient (expanded ~3.7k nodes vs A*'s 98 for the start example).
- DFS is neither optimal nor complete in practical resource-limited runs; iterative deepening DFS can be used to get optimal solutions but needs careful implementation and can still be expensive.

7. Assumptions & notes

- The blank tile is 0 and goal state is [0,1,2,3,4,5,6,7,8] as required by the assignment.
- All moves have uniform cost 1.
- Timings measured on a desktop environment (microseconds to milliseconds for these sample runs). Times vary with hardware; node counts are the more stable metric for algorithmic comparison.